mail-c

An E-mail client & server in C

Network Programming — BIT, Fall 2026

Nikolas Tan 汤铁峰 1820232065

The project

An E-mail application implementation to satisfy a course of **Network Programming** in BIT Fall 2026 semester.

- a **server** speaking **SMTP** and **POP3**
- a **client** with a simple terminal interface
- storage in **SQLite**, concurrency via a **generic thread pool**

Tech stack

- C
- SQLite
- UNIX socket API



TODO

- ✓ config file parsing (key value pair in `.txt`)
- ✓ UNIX socket programming
- ✓ implement CRLF buffering for receiving TCP messages
- ✓ generic thread pool
- ✓ CLI (enter username, password, send email)
- ✓ DB connection
- ✓ log system
- ✓ integrate accounts
- ✓ barebone SMTP protocol — client send / server reply / client parse / server parse
- ✓ POP3 protocol — USER, PASS / STAT, LIST / RETR / QUIT
- ✓ IMF construction
- ✗ gui

Highlights

- minimum dependencies (base lib)
- sane defaults
  - arena allocator
  - length-based string (**less segfaults!**)
  - per-thread log
- **no malloc**
- fast build time (**unity build**, no linking, 675 ms debug build, 4.5 s `-O3` build)
- concurrent client handling (with threadpool)

Allocations are explicit

```
String8 pop3_process_line(Arena* arena, Session* state, String8
line);
i64Array db_get_array_mail_ids(Arena* arena, sqlite3* db, String8
recipient);
```

Parsing is trivial with length-based string

```
    String8List list = str8_split(scratch.arena, line,
str8_lit("<"));
    if (list.node_count != 2) {
      reply = smtp_reply_syntax_error(arena);
      log_user_errorf("[t%u] [error] MAIL FROM command got dubious
syntax\n",
                      log_thread_id);
    } else {
      String8 sender = parse_mail_address(list.last->string);
      state->sender = str8_copy(state->arena, sender);
      log_infof("[t%u] [info] Successfully parsed sender address:
%.*s\n",
                log_thread_id, str8_fmt(sender));
```

```
        reply = smtp_reply_ok(arena);
    }
```

Challenges

- network programming misconceptions
- implementing both server and client simultaneously
- unfamiliarity with the new defaults (especially the log system)
- bad API design for the protocols
- multithreaded architecture

Showcase: logging tangled with control flow

```
log_scope_begin();
b32 res = pop3_send_and_response_ok(ctx->pop3_fd,
            str8f(scratch2.arena, "USER %.*s%.*s",
                  str8_fmt(username), str8_fmt(CRLF)),
            &ctx->recv_buf);
LogScopeResult log = log_scope_end(scratch2.arena);
append_data_to_file_path(ctx->log_path,
log.strings[LogMsgKind_Info]);
if (log_active && log.strings[LogMsgKind_UserError].size != 0) {
  append_data_to_file_path(ctx->log_path,
log.strings[LogMsgKind_UserError]);
  continue; // control flow decided by log state
}
```

Showcase: bad API design for the protocols

SMTP — a named function for each reply and command

```
String8 smtp_reply_ready(Arena* arena, String8 hostname);
String8 smtp_reply_closing(Arena* arena, String8 hostname);
String8 smtp_reply_ok(Arena* arena);
String8 smtp_reply_start_mail_input(Arena* arena);
String8 smtp_reply_service_not_avail(Arena* arena);
String8 smtp_reply_unknown_command(Arena* arena);
String8 smtp_reply_syntax_error(Arena* arena);
String8 smtp_reply_recipient_rejected(Arena* arena);

String8 smtp_command_ehlo(Arena* arena, String8 rcpt);
String8 smtp_command_from(Arena* arena, String8 from);
String8 smtp_command_to(Arena* arena, String8 to);
```

```
String8 smtp_command_data(Arena* arena);
String8 smtp_command_quit(Arena* arena);
```

Showcase: bad API design for the protocols

POP3 — format at the point of use

```
reply = str8f(arena,
    "+OK USER %.*s found in database%.*s",
    str8_fmt(username), str8_fmt(CRLF));

reply = str8f(arena, "-ERR user not found%.*s",
              str8_fmt(CRLF));
```

No reply/command helpers; only a reused predicate + round-trip.

Limitations

- Server doesn't check for IMF compatibility
- no authentication for SMTP listener (only enforced client-side)
- SQLite assumed to work properly (minimal error handling)
- no registration interface (only direct DB manipulation)
- most responses sent in bulk, ignoring protocol max length limits
- POP3 doesn't have state checks
- logging is mixed with control flow
- no relay network handling (emails to unknown users ignored)
- no password encryption (it ends up in the log too!)

CREDITS

- Ryan Fleury and Raddebugger codebasee / github.com/EpicGames/raddebugger
- Magical Bat (Ian Dvorin) / youtube.com/@Magicalbat

demo